I'm completing Software Engineering Lab 4: Vibe Coding.
Repository: **https://github.com/SETAPESU26/31_galaga**
The assignment requires fixing one deliberate bug and implementing three features using AI-assisted coding.
I want to complete this iteratively, using approximately four focused prompts, while understanding and testing every change.
For this first iteration, inspect the attached `game.py` and identify the bug in the cubic Bézier implementation that causes incorrect enemy movement.
Please:

1. Explain the bug and why it affects enemy movement.
2. Show the corrected function.
3. Make only the minimum changes required.
4. Explain how I can verify the fix.
5. Do not implement the other three features yet.

I'll apply the correction locally and test it before proceeding.

Worked for 7s

Iteration 1/4 — Fix Enemy Bézier Movement

Bug identified in game.py

I inspected the `game.py` from SETAPESU26/31_galaga ↗. The deliberate bug is in the `bezier()` function near the beginning of the file.

1. What is the bug?

A cubic Bézier curve uses four control points to create smooth movement:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3$$

Here, t ranges from 0 to 1, and the four points define the movement path.

The existing code incorrectly calculates the contribution of the third control point, `p2`.

Current incorrect expression:

Python

```
3 * u * t * p2[0]
```

Correct expression:

Python

```
3 * u * t * t * p2[0]
```

The same mistake occurs in the calculation of `y`.

The missing multiplication by `t` causes the third control point to have too much influence during movement. Consequently, enemies deviate from their intended curved paths.

For example, at `t = 0.5`, the incorrect coefficient of `p2` is `0.75`, whereas the correct coefficient is `0.375`.

2. Corrected function

Replace only the existing `bezier()` function with this version.

Python

```
def bezier(p0, p1, p2, p3, t):
    u = 1 - t
    x = u ** 3 * p0[0] + 3 * u * u * t * p1[0] + 3 * u * t * t * p2[0] + t ** 3 * p3[0]
    y = u ** 3 * p0[1] + 3 * u * u * t * p1[1] + 3 * u * t * t * p2[1] + t ** 3 * p3[1]
    return pygame.Vector2(x, y)
```

Changes made: Exactly two lines, one for each coordinate. No other game logic needs modification.

3. Visual explanation

Illustrative movement path: Incorrect vs. Correct

The orange path deviates because P2 has excessive influence. Both curves still share the same start and end points.

4. How to verify the fix

1. Open `game.py` in VS Code.
2. Run `python game.py` before applying the fix and observe the enemies entering the screen.
3. Apply the two-line correction.
4. Run `python game.py` again.
5. Confirm that the enemies follow smoother curved entry and diving paths, without the exaggerated deviation caused by the incorrect formula.

You can also verify the mathematical correctness independently with a short Python test:

</> Python

```python
from game import bezier

p0 = (0, 0)
p1 = (0, 100)
p2 = (100, 100)
p3 = (100, 0)

assert bezier(p0, p1, p2, p3, 0) == (0, 0)
assert bezier(p0, p1, p2, p3, 0.5) == (50, 75)
assert bezier(p0, p1, p2, p3, 1) == (100, 0)

print("Bézier tests passed!")
```

The midpoint assertion fails with the original implementation and passes after the fix.

5. Save your first commit

After testing locally:

```bash
git add game.py
git commit -m "fix: correct cubic Bezier enemy movement"
```

For your Lab 4 evidence, capture a screenshot of this conversation, the two-line VS Code diff, and ideally before/after recordings of enemy movement.

Iteration 1 scope: One deliberate bug, two corrected lines. The three feature functions (`enemy_tint` , `on_wave_start` , and `shield_charges`) remain untouched.

ok nice ive tested the code and it looks good to go next bug is related Implement `enemy_tint(kind)`

Called once per visible enemy per frame in `draw` , as `color = enemy_tint(enemy.kind) or ENEMY_COLORS[enemy.kind]` . `kind` is `"boss"` , `"red"` , or `"blue"` . Return an `(r, g, b)` override, or `None` to keep the default from `ENEMY_COLORS` . Idea: recolor enemies by wave number so later waves look visually distinct.

i will replace code locally and test

Iteration 2/4 — Implement `enemy_tint(kind)`

Now that the Bézier movement fix is tested, we can implement the first feature: **wave-based enemy recoloring**.

The goal is to make later waves visually distinct without changing enemy movement, health, difficulty, or any other game mechanics.

1. Implementation approach

Currently, `game.py` contains:

Python

```
def enemy_tint(kind):
    """Return an (r, g, b) colour override for an enemy kind, or None for the default."""
    pass
```

The `draw()` method already uses:

Python

```
color = enemy_tint(enemy.kind) or ENEMY_COLORS[enemy.kind]
```

However, `enemy_tint()` only receives the enemy type, not the current wave number.

Minimal solution: Add a `wave` parameter to the function and update the single call in `draw()`.

2. Updated function

Replace `enemy_tint()` with:

Python

```
def enemy_tint(kind, wave):
    """Return a wave-dependent enemy colour override."""
    if wave == 1:
        return None

    palettes = [
        {
            "boss": (90, 220, 90),
            "red": (230, 70, 70),
            "blue": (80, 140, 240)
        },
        {
            "boss": (255, 190, 60),
            "red": (255, 110, 160),
            "blue": (150, 100, 255)
        },
        {
            "boss": (60, 240, 200),
            "red": (255, 150, 60),
            "blue": (100, 230, 255)
```

This implementation cycles through four palettes.

Enemy color previews

Boss	Red	Blue
W1	W1	W1
W2	W2	W2
W3	W3	W3
W4	W4	W4

Wave 1 preserves the original colors. Waves 2–4 use alternate palettes, repeating every four waves.

3. Update the function call

Inside `Game.draw()`, find:

Python

```python
color = enemy_tint(enemy.kind) or ENEMY_COLORS[enemy.kind]
```

Replace it with:

Python

```python
color = enemy_tint(enemy.kind, self.wave) or ENEMY_COLORS[enemy.kind]
```

That's all. No other modifications are required.

4. Testing the feature

You can test the color logic without playing through several waves. Create a temporary `test_tint.py` in the same directory:

Python

```python
from game import enemy_tint
```

```
# Wave 1 uses default colors
assert enemy_tint("boss", 1) is None
assert enemy_tint("red", 1) is None

# Wave 2 introduces new colors
assert enemy_tint("boss", 2) == (255, 190, 60)
assert enemy_tint("blue", 2) == (150, 100, 255)

# Different waves use different palettes
assert enemy_tint("red", 2) != enemy_tint("red", 3)

# Palette cycles back after four waves
assert enemy_tint("boss", 5) == (90, 220, 90)

# Invalid enemy types have no override
assert enemy_tint("unknown", 2) is None
```

Run:

Bash

```
python test_tint.py
```

For visual testing, temporarily change the wave value immediately after `self.reset()` in `Game.__init__`:

Python

```
self.wave = 2
```

Launch the game and check the enemy colors. Then change it to `3` and `4` to inspect the remaining palettes.

Important: Remove this temporary wave assignment before committing, since it is only for testing.

5. Commit after verification

Bash

```
git add game.py
git commit -m "feat: add wave-based enemy color palettes"
```

This keeps the second iteration separate from your Bézier bug fix.

Progress: 2/4 iterations

- Bézier movement bug — fixed and tested.
- Wave-based enemy colors — ready for local testing.
- `on_wave_start(wave)` — not implemented.
- `shield_charges(wave)` — not implemented.

Once you've tested the color changes, we can proceed to `on_wave_start(wave)` without touching anything else.

> ok tested this as well looks good next task to debug Task 3: Implement `on_wave_start(wave)`
>
> > Called once from `spawn_wave(wave)`, right after that wave's enemy list is built — both at game start and every time a wave is cleared. It receives the 1-indexed wave number. Its return value is ignored. Idea: show a "WAVE 3" banner for a second, or ramp up the dive rate.

Iteration 3/4 — Implement `on_wave_start(wave)`

Your first two iterations are complete. For Task 3, we'll implement a **wave announcement banner** that appears for one second whenever a new wave begins.

This is a good feature for Lab 4 because it's visually testable and doesn't interfere with enemy behavior or difficulty.

1. How the feature works

The existing function is:

‹/› Python

```
def on_wave_start(wave):
    """Called at the start of every wave; add banners, speed-ups, or palette swaps here."""
```

```
    pass
```

It's called from `spawn_wave(wave)`:

```python
on_wave_start(wave)
```

The problem is that `on_wave_start()` doesn't have access to the `Game` object or drawing screen. So we'll store the latest wave announcement in a small module-level variable and let `Game.draw()` render it.

2. Change 1 — Implement the function

Replace the existing function with:

```python
wave_banner = None

def on_wave_start(wave):
    """Display a banner for one second when a new wave starts."""
    global wave_banner
    wave_banner = (wave, pygame.time.get_ticks())
```

`pygame.time.get_ticks()` gives the number of milliseconds since Pygame was initialized. This allows us to display the banner for exactly 1,000 milliseconds.

3. Change 2 — Render the banner

Inside `Game.draw()`, locate:

```python
hud = font.render(f"Score {self.score}  Lives {self.lives}  Wave {self.wave}  R = reset", True, (240, 240, 240
screen.blit(hud, (10, 8))
```

Immediately after these two lines, insert:

```python
# Display the new wave announcement for one second.
if wave_banner is not None:
    wave_num, start_time = wave_banner
    if pygame.time.get_ticks() - start_time < 1000:
        banner_font = pygame.font.Font(None, 64)
        banner = banner_font.render(
            f"WAVE {wave_num}", True, (255, 220, 80)
        )
        banner_rect = banner.get_rect(
            center=(WIDTH // 2, HEIGHT // 2)
        )
        screen.blit(banner, banner_rect)
```

The game now renders a large yellow wave announcement at the center of the screen.



Illustrative preview of the centered wave announcement

4. Testing the implementation

Test A — Initial game launch

Run:

```bash
python game.py
```

Expected behavior: `WAVE 1` appears immediately, disappears after approximately one second, and the game continues normally.

Test B — Subsequent waves

To test without defeating every enemy, temporarily add the following line immediately after `game = Game()` inside `main()`:

```python
game.enemies = []
```

On the next update, the game should advance to Wave 2 and show `WAVE 2` for one second.

Remove this temporary line after testing.

Test C — Game reset

Press `R` while playing. `WAVE 1` should appear again, and the HUD should reset to Wave 1.

5. Commit after successful testing

```bash
git add game.py
git commit -m "feat: display wave start announcement banner"
```

This implementation changes only the wave-start hook and the rendering code. It leaves the Bézier fix, enemy colors, and shield mechanics unchanged.

Lab 4 progress:

- Iteration 1 — Bézier movement bug: complete
- Iteration 2 — `enemy_tint()`: complete
- Iteration 3 — `on_wave_start()`: ready to test
- Iteration 4 — `shield_charges()`: remaining

Once you've verified the banner, we can implement the final shield feature.

ok done its working next debuging taskTask 4: Implement `shield_charges(wave)`

Called at game start and at the start of every new wave, as `self.shield = shield_charges(self.wave) or 0`. It receives the current wave number and should return an integer number of hits the player can absorb this wave before losing a ship, or `None / 0` to disable the shield. The logic that consumes a shield charge in `hit_player()` is already implemented — you only need to decide how many charges to grant. Idea: grant 1 shield charge every 3 waves.

Iteration 4/4 — Implement `shield_charges(wave)`

This is the final feature for your Software Engineering Lab 4: Vibe Coding assignment.

We'll implement a **wave-based shield system** that grants the player one shield charge every three waves.

The shield absorption logic already exists in `hit_player()`, so this change requires modifying only **one function**.

1. Understand the existing implementation

Currently, `game.py` contains:

```python
def shield_charges(wave):
    """Return how many hits the player's shield can absorb this wave, or None to disable the shield."""
    pass
```

Elsewhere, the game already calls:

```python
self.shield = shield_charges(self.wave) or 0
```

And `hit_player()` contains:

```python
if self.shield > 0:
    self.shield -= 1
    self.invulnerable = 1.0
    return
```

This means that when the player has a shield charge, the next eligible hit consumes that charge instead of costing a ship or a life.

2. Implement the feature

Replace only the existing `shield_charges()` function with:

```python
def shield_charges(wave):
    """Grant one shield charge every third wave."""
    if wave % 3 == 0:
        return 1
    return 0
```

How it works: The modulo operator `%` checks whether the wave number is divisible by three.

Wave	Shield charges
1	0
2	0
3	1
4	0
5	0
6	1
7	0

8	0
9	1

This pattern continues automatically for every subsequent wave.

3. Test the implementation

You can test the function independently without playing through multiple waves.

Create a temporary file named `test_shield.py` :

Python

```python
from game import shield_charges

assert shield_charges(1) == 0
assert shield_charges(2) == 0
assert shield_charges(3) == 1
assert shield_charges(4) == 0
assert shield_charges(6) == 1
assert shield_charges(9) == 1

print("All shield tests passed!")
```

Run:

Bash

```bash
python test_shield.py
```

Expected output:

Plain text

```
All shield tests passed!
```

4. Verify it in the game

To verify that the shield actually absorbs damage, temporarily modify `main()`.

Find:

```python
game = Game()
```

Immediately after it, insert:

```python
game.wave = 3
game.shield = shield_charges(game.wave)
print("Shield before hit:", game.shield)

game.hit_player()

print("Shield after hit:", game.shield)
print("Lives remaining:", game.lives)
```

Expected console output:

```
Shield before hit: 1
Shield after hit: 0
Lives remaining: 3
```

This confirms that the shield absorbs one hit without reducing the player's lives.

Note that this tests the damage logic directly; it does not start an actual Wave 3. Remove the temporary testing statements afterward.

5. Commit your final feature

Once you've verified everything:

Bash

```
git add game.py
git commit -m "feat: grant shield charge every three waves"
```

Lab 4 — Final progress

Implementation progress 3/4 verified, 1 ready to test

- ✓ Fix cubic Bézier enemy movement — Tested
- ✓ Implement enemy_tint() — Tested
- ✓ Implement on_wave_start() — Tested
- ◌ Implement shield_charges() — Test pending

Once the shield test passes, you'll have completed the required **one bug fix and three AI-assisted features**, with four separate commits.

For your Lab 4 submission, keep the four prompt-and-response conversations, the test evidence, the Git commit history, and your before/after gameplay recordings.